An algorithm for finding the intersection lines between two B-spline surfaces

Q S Peng

A divide-and-conquer algorithm is presented for finding all the intersection lines between two B-spline surfaces. Each surface is organized as an n-branch tree. For each intersection line, an initial point is detected after a depth-first search along one tree, ie the host tree. Extrapolation methods are then used to trace the entire length of the line, thus the line appears naturally in a continuous form. Efficiency of the algorithm is achieved by the employment of an adaptive division strategy and by the careful choice of the representation basis of the patches on both surfaces.

algorithm, B-spline surfaces, intersection, extrapolation

Although free form surfaces are widely used nowadays, particularly in industries such as aerospace, automobile and shipbuilding, a serious problem remains unsolved. It is how to find the intersection lines between these surfaces. Various methods have been tried. From the algebraic point of view it is the problem of finding the roots of high order equations. Some heuristic methods are available, such as the Newton iterative method, but they give no guarantee of finding all the intersections between two surfaces and they may fail mysteriously.

The recent development of subdivision techniques and the foundation of so-called discrete B-spline theory provide a good basis to solve the problem in a rather geometrical way. The underlying idea is very simple: both surfaces involved are repeatedly subdivided until they can finally be approximated by planar polygons. The process of finding the intersection lines is then carried out between these planar polygons, which can be handled trivially. Nevertheless, a good design of the algorithm depends strongly on the successful exploitation of the divide-and-conquer method. It is recognized that the efficiency of a divide-and-conquer algorithm is closely related to the following issues:

- the problem concerned can easily be divided
- a good conquer performance
- quick termination time

We shall follow these principles to develop a divide-and-conquer algorithm to find the intersection lines between two surfaces. A convex box is built for each patch on the surfaces. The boxes of the two surfaces are then compared with each other. If they cannot be separated, we subdivide the patches involved and reduce their convex boxes into smaller ones. The relevant theory and methods for subdivisions are introduced in the next section and attention will be concentrated on the intersection problem of two B-spline surfaces, especially bicubic B-spline surfaces.

Computational Geometry Project, School of Computing Studies and Accountancy, University of East Anglia, UK

Present address: Department of Mathematics, Zhejiang University, Hangzhou, China

THEORIES OF SUBDIVISION TECHNIQUES FOR B-SPLINE CURVES

There is known to be a correspondence between the knots on the parametric axis and the vertices of the control polygon. Suppose a new knot is inserted into the knot set, then a new control vertex will be introduced to the polygon. Meanwhile, a corresponding segment of B-spline curve is divided into two portions by the new knot with each portion corresponding to a smaller convex hull. When more and more new knots are inserted into the same interval on the parametric axis, the number of vertices of the control polygon corresponding to the curve segment will increase accordingly and each convex hull becomes smaller and smaller while the curve segment is approximated by the polygon more and more closely.

Methods for subdividing either Bézier curves or uniform B-spline curves were demonstrated by Lane and Riesenfeld in 1977[1]. Later Cohen, Lyche and Riesenfeld[2] extended the methods to the non-uniform B-spline case and concluded that all these subdivision methods can be unified under the theory of discrete B-splines. For convenience, we shall review these methods in three cases.

Bézier curves

Theorem 1: The Bézier curve $B = [P;0,1]$ can be represented by two pieces of Bézier curve defined on $[0,½]$ and $[½,1]$ respectively.

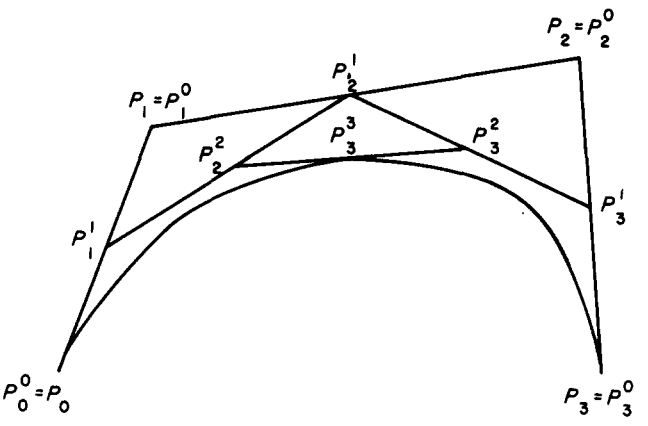

Figure 1. Bézier curve $B=[P;0,1]$ represented by two pieces of Bézier curve defined on $[0,½]$ and $[½,1]$ respectively

 0010–4485/84/040191–06 $03.00 © 1984 Butterworth & Co (Publishers) Ltd

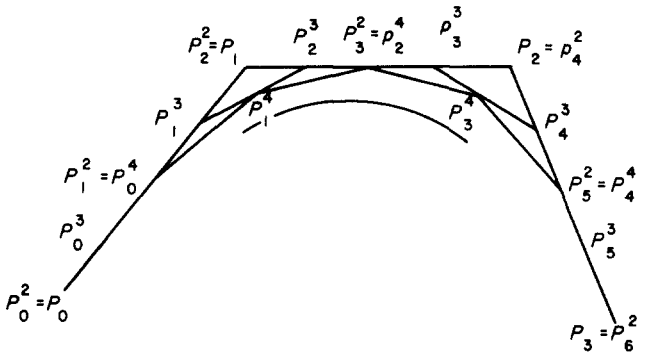

Figure 2. Theorem 2

$$B_k[\mathbf{P};0,1]\ [0,½] = B_k\ [P_0^0, P_1^1, \ldots, P_k^k;0,½] \tag{1}$$

$$B_k[\mathbf{P};0,1]\ [½,1] = B_k\ [P_k^k, P_k^{k-1}, \ldots, P_k^0;½,1] \tag{2}$$

where

$$P_i^m = \begin{cases} (P_{i-1}^{m-1} + P_i^{m-1})/2 & m = 1, \ldots, k \\ P_i & m = 0 \end{cases} \quad \text{(see Figure 1)}$$

and are produced recursively during a subdivision process.

Note that the subdivision process of a Bézier curve is exactly the same as the specification of a point on the curve using the Bézier construction method. Calculations involved are adds and shifts only.

Uniform B-spline curves

As with the division of the Bézier curve, new knots are inserted at the midpoints of the intervals. The corresponding subdivision method is shown in theorem 2.

Theorem 2: let $B\ [\mathbf{P};0,n]$ be the uniform B-spline curve.

$$B_k\ [\mathbf{P};0,n] = \sum_{i-1}^{n} P_i N_{i,k}\ (1,x)$$

where 1 in $N_{i,k}\ (1,x)$ denotes the length of each equally spaced interval, and

$$n \geqslant k \quad x \epsilon\ [(k-2)/2, n-(k-2)/2]$$

then

$$B_k\ [\mathbf{P};0,n] = \sum_{i=0}^{2n-k+2} P_i^k\ N_{i,k}\ (0.5,x) \tag{3}$$

where P_i^k is recursively defined by

$$P_i^k = (P_i^{k-1} + P_{i+1}^{k-1})/2$$

$$i=0,1,\ldots,2n-k+2, k>2$$

when

$$k = 2\ P_i^2 = \begin{cases} P_{i/2} & i \text{ even} \\ (P_{(i-1)/2} + P_{(i+1)/2})/2 & i \text{ odd} \end{cases} \quad \text{(see Figure 2)}$$

Non-uniform B-splines

The subdivision of a non-uniform B-spline curve is achieved by adding some new knots into the corresponding parametric intervals.

Theorem 3: let $(t_i)_{i=1}^{n+k}$ be the knot set on which $B_{i,k}\ (x)$ is defined and

$\{\tau_j\}_{j=1}^{m+k}$ be the new knot set with $m > n$

Then

$$B_{i,k}(x) = \sum_{j=1}^{m} \alpha_{i,k}(j)\ N_{j,k}(x)\ i=1,2,\ldots,m \tag{4}$$

where $N_{j,k}(x)$ is defined on the new knot set $\{\tau_j\}$. $\alpha_{i,k(j)}$ is the so-called discrete B-spline function which takes value only at the integers $j=1,2,\ldots,m$, and

$$f(x) = \sum_{i=1}^{n} P_i\ B_{i,k}(x) = \sum_{j=1}^{m} d(j)\ N_{j,k}(x) \tag{5}$$

with

$$d(j) = \sum_{i=1}^{n} P_i\ \alpha_{i,k}(j) \tag{6}$$

From equation (6) we see that $d(j)$, $(j=1,2,\ldots,m)$, consist of a sequence of discrete points. We will call it a d-spline 'curve' for short. Further analysis indicates that the d-spline sequence takes its corresponding continuous B-spline curve as its limit when it becomes denser and denser. This is a very important property which ensures the convergence of our subdivision process.

Mathematical manipulation yields

$$d(j) = \sum_{i=\mu-k+r}^{\mu} P_{i,j}^{[r]}\ \alpha_{i,k-r+1}(j) \quad (t_\mu \leqslant \tau_j < t_{\mu+1})$$

where

$$P_{i,j}^{[1]} = P_i,$$

$$P_{i,j}^{[r+1]} = [(\tau_{j+k-r}-t_i)\ P_{i,j}^{[r]} + (t_{i+k-r}-\tau_{j+k-r})\ P_{i-1,j}^{[r]}]$$

$$/(t_{i+k-r}-t_i) \tag{7}$$

Thus taking $r = k$, we have

$$d(j) = P_{\mu,j}^{[k]}\ \alpha_{\mu,1}(j) = P_{\mu,j}^{[k]}$$

This leads to the formation of the so-called Oslo algorithm[2,3] which can be used to calculate any point of the d-spline sequence by recursive evaluations of the above equations. The Oslo algorithm is so general that all the subdivision methods discussed so far are included and treated as special cases. However, algorithms in general forms are not always efficient. In specified circumstances, it is possible to derive a more ingenious algorithm for particular usage. For this reason, we shall introduce another pragmatic method[4] in which only one knot is inserted into the parametric interval each time.

Let $\{\mathbf{t}\}$ be the knot set. When a new knot t^* is added into the interval $\{t_i, t_{i+k}\}$, we get a new knot set $\{\tau\}$. It can be shown that

$$N_{i,k}(t) = \frac{t^*-\tau_i}{\tau_{i+k}-\tau_i} N^*_{i,k}(t) + \frac{\tau_{i+k+1}-t^*}{\tau_{i+k+1}-\tau_{i+1}} N^*_{i+1,k}(t) \quad (8)$$

where $N^*_{i,k}(t)$ is defined on $\{\tau\}$. Thus:

$$\Sigma P_i N_{i,k}(t) = \Sigma P_i \left[\frac{t^*-\tau_i}{\tau_{i+k}-\tau_i} N^*_{i,k}(t) + \frac{\tau_{i+k+1}-t^*}{\tau_{i+k+1}-\tau_{i+1}} N^*_{i+1,k}(t) \right]$$

$$= \Sigma \left(P_i \frac{t^*-\tau_i}{\tau_{i+k}-\tau_i} + P_{i-1} \frac{\tau_{i+k}-t^*}{\tau_{i+k}-\tau_i} \right) N^*_{i,k}(t)$$

let

$$\alpha_i = \frac{t^*-\tau_i}{\tau_{i+k}-\tau_i} \quad (9)$$

Then

$$\Sigma P_i N_{i,k}(t) = \Sigma [P_i \alpha_i + P_{i-1} (1-\alpha_i)] N^*_{i,k}(t)$$

$$= \Sigma d_i N^*_{i,k}(t) \quad (10)$$

where $d_i = P_i \alpha_i + P_{i-1}(1-\alpha_i)$ $i=0,1,\ldots,n$ represents the new polygon (see Figure 3). Note that when

$t^*=\tau_{i+k}$, $\alpha_i=1$ $1-\alpha_i=0$ $d_i=P_i$

$t^*=\tau_i$, $\alpha_i=0$ $d_{i+k}=P_{i+k-1}$

This means that, except at the two end points, each vertex of the new polygon within $[t_i, t_{i+k}]$ will be a linear combination of the two corresponding adjacent vertices of the original polygon. In the case that a multiple knot is inserted, more original vertices will remain in the new polygon and the calculation for the new vertices will be reduced further.

A PRACTICAL APPROACH FOR SUBDIVISION

The convex hull seems to be a natural choice of a comparison box. Using the subdivision methods stated above we can easily divide a convex hull into two smaller hulls, each of which defines one portion of the curve. We still need to consider how to 'conquer' efficiently.

As is well known, a knot point on the cubic B-spline curve is defined by three adjacent vertices and each segment

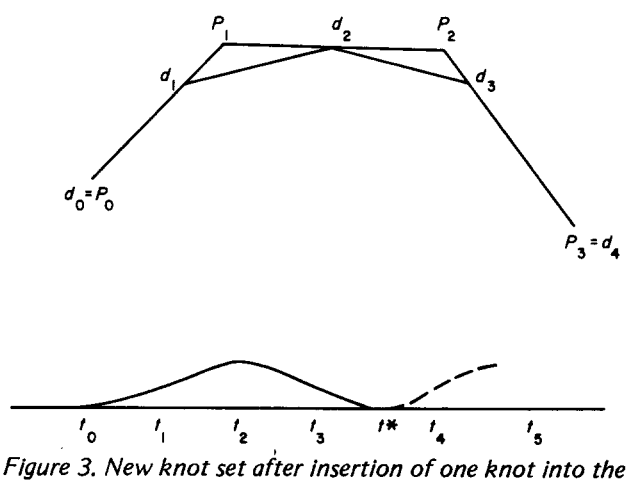

Figure 3. New knot set after insertion of one knot into the parametric interval

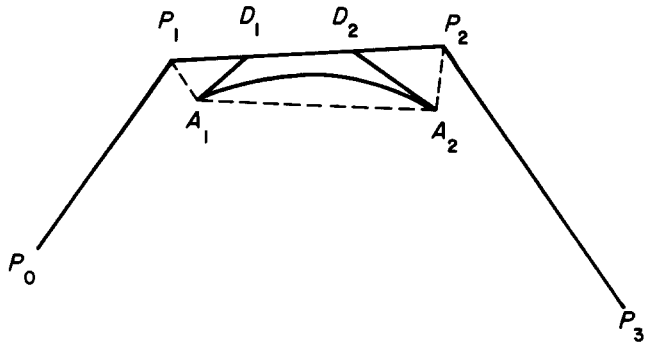

Figure 4. Portion of B-spline curve contained in a convex box made up of two knots and their corresponding vertices

of curve within the convex hull is also contained by the other two neighbouring convex hulls. In other words, the adjacent convex hulls always overlap. This means that once two rival convex hulls intersect there will be two groups of convex hulls intersecting each other at the same time. Because of this, many more comparisons than we expect have to be carried out. The overlap also implies that the convex hull is too large to be chosen as the comparison box. Large comparison boxes lead not only to poor conquer performance but also slow termination time during the divide-and-conquer process (as the thickness of a box is usually chosen as the criterion of its convergence test). Is it possible to find a box which surrounds only one span of the spline curve each time? The question is answered by Theorem 4[5].

Theorem 4: A portion of cubic B-spline curve between any two neighbouring knots is uniquely contained in a convex box made up of the two knots and their corresponding vertices (Figure 4).

A further smaller convex box which still contains the portion A_1A_2 can be built as follows. Construct a straight line A_1D_1 which is tangent to A_1A_2 at A_1 and intersects P_1P_2 at D_1, construct A_2D_2 analogously. It can be shown[6] that this newly-formed convex hull $A_1D_1D_2A_2$ is just the corresponding Bézier polygon of spline curve A_1A_2.

Thus, we have two choices in developing our algorithm to find the intersection. The first is to insert one new knot at the midpoint of the desired parametric interval, as shown in Figure 5. Three new vertices are introduced each time. Since each of the α_i ($i=j+1, j+2, j+3$) may take an arbitrary

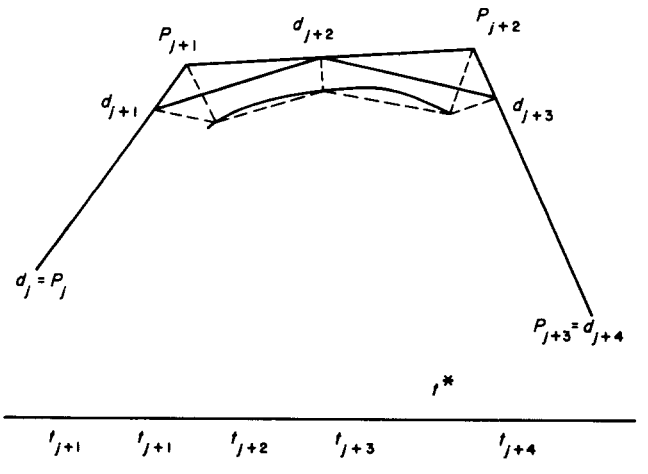

Figure 5. One new knot inserted at the midpoint of the desired parametric interval

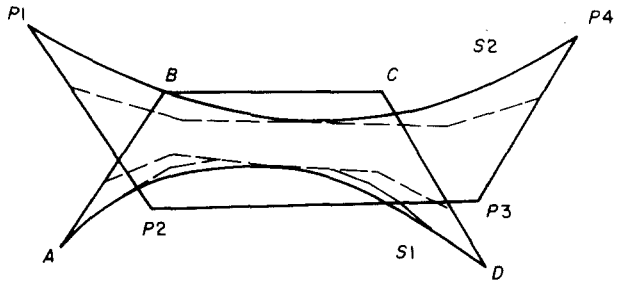

Figure 6. Contrast between static and adaptive division in 2D case

value between 0 and 1, at least nine multiplications/divisions are involved in the evaluation (according to equations (9) and (10)). To obtain the two improved comparison boxes suggested by theorem 4 we also have to calculate the pseudo-knot on the curve after each insertion. In the case of dividing a patch of a B-spline surface the number of the pseudo-knots to be evaluated will increase to five. This would cause even more calculations than producing the new control vertices.

The other choice is to add multiple knots at both ends of the parametric interval so as to redefine the portion of the spline curve in a Bézier form. Although at the beginning some extra work has to be done for the transformation, in the following subdivision process the operations involved are only additions and shifts which are easy to implement. Each time the piece of curve is split into two portions with the splitting point obtained automatically. Also notice that the Bézier convex hull of a segment of spline curve is even smaller than the improved B-spline comparison box. We thus suggest that the Bézier form be chosen as the basis for the subdivision process.

For simplicity, the discussion so far is limited merely to B-spline curves. The conclusions obtained are equally significant for B-spline surfaces, however.

THE ALGORITHM FOR FINDING THE INTERSECTION LINES BETWEEN TWO B-SPLINE SURFACES

Now we are ready to establish our algorithm.

The first step is to calculate all the knots on both surfaces, build the comparison box for each of the patches and compare the patches in pairs to see if any two boxes on the two surfaces intersect. The technique of the minimax enclosing box test is used so that we can take full advantage of the cheap division method by simplifying the conquer process. Those patches that cannot be separated from their rivals in the first round are transformed into their corresponding Bézier forms by inserting sufficient multiple knots at both the ends of parametric intervals and are then in a suitable form for further subdivision.

In order to find the intersections between the two surfaces as soon as possible, the depth-first searching method is adopted. We start our process from a transformed patch of one surface (which is referred to as the host surface). After each subdivision, four new subpatches are formed, each of which inherits all its father's rivals and after a series of comparisons each son builds its own rival list. Those subpatches whose boxes do not intersect any box on the other surface will no longer be considered.

Obviously, a quad-tree is the most suitable data structure to be used here with both the subdivision process and the topological relation between adjacent subpatches clearly represented. During the search the leftmost son of the host tree is always dealt with first. Either it or one of its rivals may be divided depending on the size of their corresponding convex hulls. The reason for the adaptive divisions on both subpatches of the two surfaces is obvious: the convex hulls of the two patches can be separated more quickly than otherwise if the two patches have no intersection at all.

A contrast between static division and adaptive division in the 2D case is shown in Figure 6. Part of curve S1 which is defined by polygon $ABCD$ is inside the other polygon $P_1P_2P_3P_4$. If we use the static method, ie, continue the division on one curve, say, $S1$ first, then there will always be some convex hulls which cannot be separated from their common rival box $P_1P_2P_3P_4$. The division has to go further and further until a termination condition is finally satisfied. On the other hand, the separation of the two curves can readily be detected just after one subdivision on each curve if the adaptive method is employed. In the 3D case, a similar judgement applies.

When both the thicknesses of the convex hulls of a pair of subpatches are less than a given tolerance, they can be treated as two planar quadrilaterals and a direct intersection test can proceed. If they do not cross each other, then the host tree is searched again. Otherwise, a piece of intersection line of the two surfaces is declared and the next pair of subpatches of the two surfaces that will form the following portion of the intersection line are found easily by using the adjacency relation between the subpatches. The process of 'divide–intersect–stretch–divide again' is repeatedly carried on until one end of the intersection line is reached. Control is then returned to the original portion and the intersection line is traced in the other direction.

It should be mentioned that when the intersection line stretches, the two adjacent quadrilaterals may not share common edges in the case where their corresponding vertices are related to different parametric values (Figure 7). This may happen when the two adjacent subpatches result from different levels of subdivision. However, according to the termination condition the distance from any point on either edge to the surface is less than the given tolerance. We can modify the next quadrilateral so that it will share a common edge with the last one. By this approach, an inter-

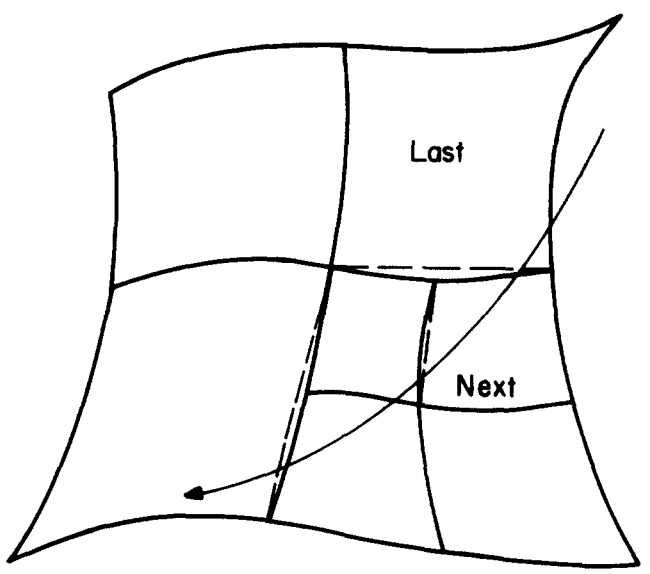

Figure 7. Two adjacent quadrilaterals may not share common edges where their corresponding vertices are related to different parametric values

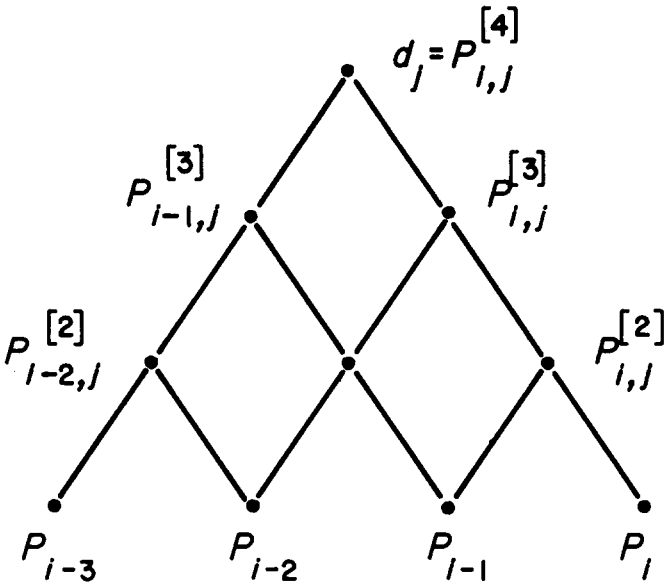

Figure 8. Result of formula (7) executed 24 times

section line will always appear in a continuous, consecutive form.

The search of the remaining convex hulls is continued until each has been checked. In this way no intersection line between the two surfaces can be missed.

FURTHER DISCUSSION

Transformation of a B-spline patch into its corresponding Bézier form

For convenience, we will discuss the matter in the 2D case. Extension to the 3D case is straightforward.

If we need to redefine a segment of B-spline curve in the Bézier form, then multiple knots should be inserted at both the ends of the parametric interval, and consequently, a new polygon will be formed (as shown in Figure 4). To evaluate the four new vertices, we may either use the Oslo algorithm to obtain the result in one pass or take a pragmatic approach by adding the multiple knots one by one. From the theoretical point of view, the first choice might be attractive. The four new vertices are evaluated one by one, each taking eighteen multiplications/divisions by implementing the recursive formula (7) six times. To obtain the whole polygon, the total number of executions of equation (7) will be 24.

We quickly recognize that the result (Figure 8) can be considerably improved if the successive order of the four new vertices is noticed. As shown in Figure 9, we can first calculate the six bottom vertices $P^{[2]}_{i,j}$ successively, then calculate the five vertices $P^{[3]}_{i,j}$ in the middle row using the results obtained in the first pass. Finally we evaluate the four upper vertices in the third pass. The total number of executions of equation (7) is now reduced to fifteen. Nevertheless, we still did not make use of the advantages emerging from inserting the multiple knots. It is shown[5] that by adding the multiple knots one by one only six new vertices (including the intermediate vertices) need to be evaluated, each taking three multiplications/divisions by implementing equation (10) once. This leads to a good saving of almost 25%. We then conclude that the pragmatic method of inserting new knots one by one is preferred in our algorithm.

Determining whether two planar convex hulls intersect each other

The convex hull which defines a subpatch within it consists of sixteen vertices. We take any three of the four vertices lying on the surface to construct a plane. The distance from the other vertices to the plane can be measured. When the maximum distance is less than a given tolerance, a convex hull could be called a planar hull and a corresponding subpatch within it can be approximately represented by a quadrilateral whose corners are the four vertices mentioned above. Then we need only to check if the two quadrilaterals intersect.

The test can be considered in two steps. First, compare each edge of one quadrilateral with the plane in which the other quadrilateral lies approximately (defined by three of its four vertices). Only if one edge intersects the plane, should we continue to the next step (if the edge is represented by parametric equations formed by its two end points the condition is then identical to the parametric value of the intersection point lying within the range [0,1]).

Second, check if the intersection point found is within the other quadrilateral. Since the plane in which the quadrilateral lies is an arbitrary plane in space, some coordinate transformations would be involved if we check the condition directly. To simplify the test, we can choose one of the coordinate planes as the test plane on which the quadrilateral has its maximum mapping area. It is then sufficient to check if the mapping of the intersection point is inside the mapping area of the quadrilateral on the test plane.

Establishing a priority among the edges to be compared with another quadrilateral

Since each edge of the quadrilateral may intersect another quadrilateral it might be necessary to carry out all the tests until the real intersection point is found. It is then wise to set a priority order among the edges so that the edge which is most likely to intersect the other quadrilateral will be checked first.

Let the parametric interval of a surface be the rectangle (U_1, U_2, V_1, V_2), as shown in Figure 10. The rectangle is divided into many small ones during the division, each representing a subpatch defined on it. The line connecting all the intersection points on the grid is a mapping of the intersection curve between the two surfaces onto the parametric plane. Let it be a *uv* line. When the entire surface is approximated by many planar sections, the segment of *uv* line within each small square is nearly linear. Extrapolate the *uv* line in the last square as if it were a straight line segment and find the intersection point, say P^*, between it and the adjacent square. The edge which contains P^* is obviously our first choice. Since the *uv* line is not actually linear, the edge which is nearest to the assumed intersection point P^* becomes our second choice. In this way, an order of priority among the edges can be determined. Notice that the grid on the parametric plane consists only

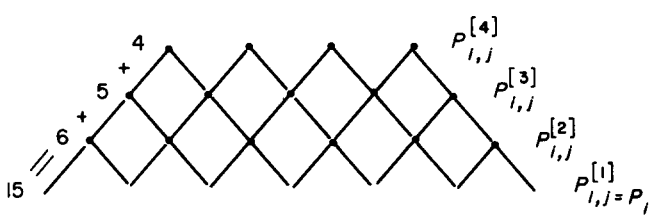

Figure 9. Figure 8 improved

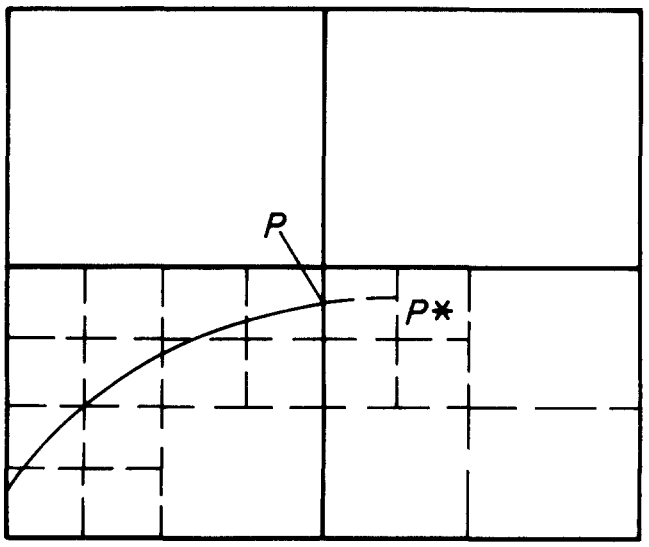

Figure 10. Rectangle (U_1, U_2, V_1, V_2)

of rectangles; it is therefore very easy to find the intersection point of the *uv* line and the neighbouring square.

From closer observation we can also see that if one edge of a quadrilateral intersects the other quadrilateral, the node in a tree corresponding to the adjacent square will certainly be on the rival list of the node (in the other tree) corresponding to the other quadrilateral and vice versa. The use of this principle can avoid a lot of unnecessary testing.

A PRACTICAL EXAMPLE

Colour plate 1 shows the intersection lines between two non-uniform B-spline surfaces.

CONCLUSIONS

We have succeeded in developing an algorithm to find the intersection lines between two B-spline surfaces. The algorithm differs from many others[7,8] in the following ways:

- The algorithm is strongly based on the divide-and-conquer technique. For each subpatch of the surfaces, there is a rival list associated with it. During the subdivision process, the rival information is inherited and revised automatically. Only those subpatches whose boxes cannot be separated from that of their rivals will be subdivided further. Meanwhile, the depth-first searching method is used in order that the intersections between two surfaces can be detected as soon as possible. The employment of the adaptive division strategy and the careful choice of the basis for the patches of both surfaces avoid a lot of unnecessary divide-and-conquer steps and therefore enable the algorithm to work more efficiently.
- Once a portion of the intersection line between the two surfaces is found, the algorithm will run in another mode. Using the coherence property of the surfaces, we simply extrapolate the intersection obtained until the entire length is traced. The intersection line appears naturally in a continuous form rather than as a collection of separated pieces, avoiding the need for post-sorting. Surfaces with slope-discontinuities (defined by multiple knots or vertices) can also be coped with trivially.
- An overall search follows after an intersection line is obtained. This is necessary in order not to miss any possible intersections between the two surfaces. The process is rather expensive, however, compared with the simple tracing of an intersection line. It may sometimes be desirable to exit from the algorithm as soon as the intersection curves are evaluated, provided that the general nature of the intersection between the two surfaces has been well understood. The fact that an intersection curve is always obtained first in implementing the algorithm provides such an opportunity. Some heuristic improvement can be achieved[5].

ACKNOWLEDGEMENTS

The author is grateful to Professor A R Forrest for his support and direction. Colour pictures were produced in the Computational Geometry Laboratory, University of East Anglia. Thanks are also given to Professor R X Tang for his valuable advice in forming the algorithm.

REFERENCES

1 **Lane, J M and Riesenfeld, R F** 'A theoretical development for the computer generation of piecewise polynomial surfaces' *IEEE Trans. Pattern Anal. and Mach. Intelligence* Vol PAMI-2 No 1 (January 1980)

2 **Cohen, E, Lyche, T and Riesenfeld, R** 'Discrete B-spline and subdivision techniques in computer-aided geometric design and computer graphics' *Comput. Graphics Image Proc.* Vol 14 No 2 (October 1980)

3 **Riesenfeld, R F et al** 'Using the Oslo algorithm as a basis for CAD/CAM geometric modelling' *Proc. Second Ann. NCGA National Conf.* National Computer Graphics Association, Baltimore, MD, USA (June 1981)

4 **Boehm, W** 'Inserting new knots into B-spline curves' *Comput.-Aided Des.* Vol 12 No 4 (July 1980)

5 **Peng, Q S** *Volume modelling for sculptured objects* Ph D thesis, University of East Anglia, UK (September 1983)

6 **Boehm, W** 'Generating the Bézier points of B-spline curves and surfaces' *Comput.-Aided Des.* Vol 13 No 6 (November 1981)

7 **Carlson, W E** 'An algorithm and data structure for 3D object synthesis using surface patch intersections' *Comput. Graphics* Vol 16 No 3 (July 1982)

8 **Palmer, T R** *Sculptured surfaces in solid object modelling systems* Ph D thesis, University of East Anglia, UK (December 1982)

BIBLIOGRAPHY

Forrest, A R 'Interactive interpolation and approximation by Bézier polynomials' *Comput. J.* Vol 15 No 1 (January 1972)

Forrest, A R *User interfaces for free-form surface design* Tutorial notes, ACM SIGGRAPH '82, Boston, MA, USA (July 1982)

Riesenfeld, R F *Applications of B-spline approximation to geometric problems of computer-aided design* Ph D thesis, Syracuse University (1973)

Su, B and Liu, D *Computational geometry* Fu Dan University, Shanghai Science and Technology Press, Shanghai, China (January 1981)